Next GM V2 Architecture and Data Model Audit

Audit-only report generated from the Codex review of the current Next GM V2 Sandbox app. Scope: architecture, data model, persistence, type safety, relationships, event history, and database readiness. No source files were modified.

1. Executive Summary

Blunt answer: the app is playable and meaningfully typed, but it is not yet architecturally strong enough for a deep long-running GM simulation without model hardening first. It has a real domain core in src/game, durable localStorage saves, stable wrestler/title/rivalry IDs in many places, and solid deterministic weekly resolution. The weak point is that the save is a large nested runtime snapshot, App.tsx still owns too much domain mutation, segment/result models are broad optional bags instead of discriminated unions, and the game records several histories as screen-specific recap structures rather than one durable event ledger.

Area	Estimate	Verdict
Architecture readiness	62%	Mostly modular, still UI-orchestrator heavy
Data model completeness	68%	Good v1 fields, incomplete deep-sim history/contracts/events
Relationship modeling quality	70%	IDs mostly present, names duplicated for display
Persistence readiness	58%	Works for localStorage, weak versioned migration story
Database migration readiness	42%	Would need normalization pass before SQLite/Postgres
Type safety confidence	72%	Strong base types, weak unions/casts/optional bags
Week-to-week simulation scalability	60%	Good deterministic loop, history/event pressure will bite

Verification run: npm test -- --run passed 21 files / 115 tests. npm exec tsc -- --noEmit passed. npm run build passed, with Vite warning that the JS chunk is 1,183.63 kB and CSS is 434.00 kB.

2. Repo Structure Reality Check

Area	Reality
UI shell	src/App.tsx, src/components, src/screens, src/booking, src/roster, src/social
Domain/sim	src/game/types.ts, scoring.ts, advanceWeek.ts, market.ts, cpuRivalLoop.ts
Static data	data/finance, data/rosters, catalogs in src/game/*Catalog.ts
Persistence	localStorage save index in src/gameStorage.ts; runtime normalization in src/game/migration.ts
Main state owner	App() holds game, screen, active save, profile/focus state, and many mutation handlers
Main model owner	GameState owns nearly everything as nested arrays

Classification: mostly modular with major UI-domain leakage. There is no obvious circular dependency, but App.tsx has excessive fan-out and remains the command bus.

```
Dependency shape: App.tsx -> game, booking, screens, roster, social, gameStorage. Booking/screens/roster/social
all depend on src/game. src/game depends on static data/catalogs and migration/save model types.
```

3. Core Entity Inventory

Entity/model	File/type	Key fields	Created/updated	Persisted	Notes
Save record	gameStorage.ts StoredSaveRecord	id, name, createdAt, lastPlayedAt, state: unknown	createSaveRecord/updateSaveRecord	Yes	Save wrapper version only, not domain schema version
Saved runtime state	migration.ts SavedGameState	game, screen, profileReturnScreen, profileWrestlerId	buildSavedGameState/persistGameSnapshot	Yes	Persists navigation alongside domain
Game state	types.ts GameState	season/week, GM/brand, wrestlers, titles, rivalries, histories, social, finance, market, CPU rivals, currentShow	createNewGame/runShow/advance/handlers	Yes	Single nested aggregate

Entity/model	File/type	Key fields	Created/updated	Persisted	Notes
GM/brand	GameState + RivalGMAssignment	gmName, gmStyle, brandName, brandStyle, rival assignments	setup/new game	Yes	Player brand lacks stable brandId; market uses player owner string
Wrestler/talent	types.ts Wrestler	id, name, stats, sentiment, record, TV time, injury	Top 200, cloneWrestlers; updated by show/advance/market/profile	Yes	Good core; contract lives separately
Contract	types.ts MarketContract	id, wrestlerId, owner, weeks, salary, status, payment	createMarketContract; sign/renew/trade/advance	Yes	Market contract more than full employment model
Booking segment	types.ts Segment	id, type, participantIds, winnerId, championshipId, rivalryId, stipulationId, catalogId	App, BookingScreen, smart rundown; updated by handlers	Yes as currentShow	Needs discriminated union
Show result	types.ts ShowResult	id, season/week, score, segment results, notes/events, fallout	runShow append	Yes in showHistory	Good weekly artifact, not universal event ledger
Segment result	types.ts SegmentResult	segmentId, type, ids/names, score, deltas, links	runShow append	Yes	Uses both IDs and duplicated names
Fallout	types.ts LockerRoomFallout	morale, overuse, underuse, injury, title stat notes	runShow	Yes in result	Partly structured, partly note text
Injury	Wrestler fields + fallout/recovery notes	status, description, weeks remaining, occurred week	evaluateInjuries/recoverWrestlerInjury	Yes	No separate injury event lifecycle
Rivalry	types.ts Rivalry	id, name, participants, structure, heat/freshness/status/stakes	seed, App create, crowd spark; scoring/advance/end	Yes	Strong IDs, weak side/team model
Championship	types.ts Championship	id, name, division, catalog, championIds, contenders, prestige, defenses	createDefaultChampionships; show/App title actions	Yes	Good title aggregate
Finance report	types.ts FinanceReport	attendance, revenue/cost lines, profit, modelVersion	generateFinanceReport append	Yes	Strong report, weak show/result FK
Social/IWC post	types.ts SocialPost	id, week/season/show, category/tone, related IDs	generateSocialPosts and AI commentary path	Yes	Text is durable artifact; needs result/segment refs
Market	types.ts MarketState	contracts, transactions, cooldowns, office mandate, weekly board	createDefaultMarketState; market/advance	Yes	Good bounded market model
CPU rival brand	types.ts RivalBrandState	id, roster IDs/state, titles, rivalries, finance, results, budget	createRivalBrandUniverse; CPU draft/week/market	Yes	Parallel but lighter sim
Week Review	weekReviewScreenReads view model	fallout groups, story events	derived from result/game	No	Screen read model only
Season archive	types.ts SeasonArchiveSummary	best show, stars, title/rivalry highlights, champions snapshot	buildSeasonArchiveSummary at transition	Yes	Summary-only; loses detailed event linkage

4. Relationship Map

Relationship	Modeling approach	IDs or strings	Risk
Game -> wrestlers/titles/rivalries/currentShow/results	Nested arrays in GameState	IDs inside records	Medium: hard to query/migrate
Wrestler -> contract	MarketContract.wrestlerId	ID	Low
Player brand -> roster	GameState.wrestlers only	Implicit ownership	Medium: no brandId
CPU brand -> roster	rosterWrestlerIds plus rosterState	Duplicated IDs/state	Medium: can drift
Segment -> participants	participantIds	IDs	Low
Segment -> rivalry/title/stipulation/catalog	optional IDs	IDs	Low/medium: invalid combos possible
ShowResult -> SegmentResult	nested array	segment IDs	Low

Relationship	Modeling approach	IDs or strings	Risk
SegmentResult -> planned Segment	segmentId plus copied fields	ID + duplicated snapshot	Medium
Rivalry -> participants/history	participantIds and rivalryHistory	IDs + names	Medium duplication
Championship -> champion/history	championIds and championshipHistory	IDs + names	Low/medium
Finance report -> show	id pattern and week/show fields	Implicit by week/name	Medium
Social post -> events	related wrestler/rivalry/title IDs	Partial IDs	Medium: no resultId/segmentId
Calendar week -> result	CalendarWeek.resultId set on advance	ID	Medium: only after advance
CPU result -> week	nested under rival brand by season/week	Implicit by week	Medium

5. ID and Reference Audit

Strong ID usage: wrestlers, championships, rivalries, contracts, segment participants, social related entities, and deterministic show result IDs are mostly stable. Championships come from catalog/canonical IDs; rival brand IDs are deterministic from brand names; show results use season/week IDs.

Fragile generated IDs: save IDs use crypto.randomUUID or Date.now/Math.random fallback, which is fine for save records. Gameplay IDs are riskier: booking segments, smart rundown segments, social inbox TV segments, player-created rivalries, and manual title assign/revoke events use Date.now().

String relationship risk: participantNames, wrestlerName, championshipName, rivalryName, showName, and brandName are useful historical snapshots, but they become risky if treated as identity. Social generation finds wrestlers by biggestMomentumGain.name, which should be an ID.

Must fix before deeper simulation: deterministic domain IDs for booking segments and player-created rivalries; resultId/segmentResultId references on finance/social/history; names treated only as display snapshots.

6. State Ownership and Data Flow Audit

Current flow: App.tsx React state -> booking mutation handlers -> GameState.currentShow -> runShow(game) -> ShowResult plus wrestler/title/rivalry/finance/social/CPU updates -> persistGameSnapshot -> Results -> Week Review -> advanceGameWeek -> currentShow reset, recovery/rivalry decay/CPU/market/office advance -> save/load through migration.

State	Owner
Runtime game	useState<GameState | null> in App.tsx
Navigation/profile/focus	React state plus saved screen/profileWrestlerId
Save storage	gameStorage.ts raw JSON; migration.ts normalization
Booking UI view model	buildBookingModel.ts derived model
Results/week/finance/social reads	screen read modules and gameContextReads.ts
Domain simulation	scoring.ts, advanceWeek.ts, market.ts, cpuRivalLoop.ts

Main leakage: App.tsx still creates and updates segments, titles, rivalries, saves, market actions, and season archive. Some read-model logic is duplicated between App.tsx and extracted read modules. runShow is a large transaction boundary that calculates results and assembles updated game state.

7. Persistence and Save Model Audit

Persisted: entire SavedGameState, including game, screen, optional profile state, and all nested GameState arrays. Save index metadata lives under next-gm-save-index with version 1.

Not persisted separately: no domain schema version, no event log table, no normalized entities, no separate UI state beyond profile return/id. Booking selected segment is transient.

Good: migration has broad normalizers for wrestlers, market, CPU, current show, social inbox, season archives, and legacy budget normalization. Weak: migration is ad hoc shape normalization, not versioned migrations. SaveIndex.version is wrapper version, not game schema version. Moving to SQLite/Postgres would require splitting most of GameState.

8. Data Model Completeness By System

System	Existing fields	Missing / overloaded / storage concern
Booking/card	Segment with type, participants, title/rivalry/stipulation/catalog/duration	Needs discriminated union; team assignments overloaded by participant order for M020
Show results	ShowResult + SegmentResult with scores, deltas, notes, title/rivalry events	Needs downstream resultId/segmentResultId; result type should vary by segment kind
Wrestler state	stats, sentiment, record, TV time, injury	Missing brand membership row, contract pointer, chemistry, weekly snapshots
Fatigue/morale/momentum	numeric fields updated on show/advance	Needs wrestler_weekly_state history for trends
Injuries	current status fields + fallout/recovery notes	Needs occurred/worsened/protected/cleared lifecycle events
Rivalries	participants, structure, heat/freshness/status/stakes, pending end	Needs explicit sides/team roles, payoff target, durable beat links
Championships	champion IDs, contenders, prestige, defenses, history	Needs title_reign entity separate from current title state
Finance	v3 report, breakdown lines, attendance, costs/revenue	Needs showResultId, venue/market IDs, contract transactions as cost lines
Social/IWC	posts with category/tone/related IDs	Needs post-to-result/segment links; text too durable as source data
Market	contracts, transactions, weekly board, cooldowns	Good v1; no full employment/scouting history
CPU rivals	roster state, contracts, titles, rivalries, weekly results	Useful pressure sim; not same schema as player brand
Office pressure	mandate state/history	Should link mandate events to finance/show causes
Week Review	derived view model	Fine as screen-only if no separate review gameplay
Season history	archive summary + show/title/rivalry/finance histories	No unified history ledger
Settings/difficulty	difficulty profile + budget/draft mode	Good; no separate campaign settings version

9. Event and History Model Audit

The app records several structured histories, but not a unified event model. Good: showHistory persists full ShowResult; championshipHistory and rivalryHistory persist typed events; financeReports, socialPosts, marketState.transactions, mandateHistory, CPU weekly results, and injuryRecoveryNotes persist. Cause/effect is partly reconstructable for recent shows via segment results and history events.

Not good enough for deep simulation: there is no common HistoryEvent shape with eventType, source, resultId, segmentId, affected entity IDs, deltas, and display payload. Finance reports lack showResultId; social posts lack resultId/segmentId; injury history is split between result fallout and recovery notes; many explanations are note strings.

Verdict: historical screens are supported at v1 level. Trend charts and two-month-later causality will become fragile without a week/event ledger.

10. Architecture Boundary Audit

Boundary	Status	Strong files	Weak files	Recommendation
UI vs domain	Leaky	extracted screens/read modules	App.tsx	Move mutations into domain command helpers
Fixtures vs state	Mostly clean	seed.ts, catalogs, data/*	Top 200 cast	Add runtime validation for generated data
Simulation vs rendering	Mixed	scoring.ts, advanceWeek.ts	social text and result assembly intertwined	Return event/result object; render text downstream
Save/load vs runtime	Mixed	gameStorage.ts, migration.ts	no game schema version	Add saveVersion and migration registry
Player summaries vs hidden state	Mostly clean	gameContextReads.ts, screen reads	duplicated helpers in App.tsx	Consolidate selectors/read models
CPU sim vs player sim	Parallel/asymmetric	cpuRivalLoop.ts	different CPU/player schemas	Normalize shared brand roster/week result concepts

Boundary	Status	Strong files	Weak files	Recommendation
Social renderer vs event data	Weak	SocialPost related IDs	social as final text	Add fan reaction event object before post text

11. Type Safety Audit

Strong: central shared domain types in types.ts; good string unions for screen, segment type, show type, difficulty, market states, rivalry states, and injury states; broad migration normalization.

Weak: Segment and SegmentResult are optional-field bags. A Promo can technically carry winnerId; an Open Challenge can look like a match with missing opponent until resolution. Wrestler alignment/division/roleTier and similar fields are weak strings. JSON parsing and migration use casts that hide validation gaps. Duplicate read types remain in App.tsx and extracted modules.

Booking-specific verdict: booking segments and segment results should become discriminated unions before adding more segment types, finishes, match rules, or rivalry-specific outcomes.

12. Conceptual Normalized Schema

Table/collection	PK	Important fields	Current equivalent	Migration gap
saves	save_id	name, dates, schema_version	StoredSaveRecord	Add domain version
seasons	season_id	save_id, number, starting/final money	GameState season + archives	Split from snapshot
weeks	week_id	season_id, week_number, show_name/type, completed	CalendarWeek	Result FK already partial
brands	brand_id	name, style, is_player	player fields + rivalBrands	Make player brand first-class
gms	gm_id	name, style, brand_id	GM fields/rival assignments	Separate GM identity
wrestlers	wrestler_id	identity/source/static stats	Wrestler/Top 200	Split static from career state
brand_roster_members	membership_id	brand_id, wrestler_id, acquired/released weeks	player wrestlers + CPU roster IDs	Normalize ownership
contracts	contract_id	wrestler_id, owner brand, weeks, salary, status	MarketContract	Good conceptual fit
championships	championship_id	brand_id, catalog_id, prestige, division	Championship	Mostly ready
title_reigns	reign_id	championship_id, champion IDs/group, start/end, defenses	current title + history	Needs reign entity
rivalries	rivalry_id	brand_id, structure, status, stakes, heat/freshness	Rivalry	Add sides/roles
rivalry_participants	composite	rivalry_id, wrestler_id, side/order	participantIds	Participant order semantics
booking_cards	card_id	brand_id, week_id, status	currentShow	Create card entity
booking_segments	segment_id	card_id, type/catalog/stipulation/duration	Segment	Stable IDs + union fields
booking_segment_participants	composite	segment_id, wrestler_id, side/order/role	participantIds	Explicit teams/sides
shows	show_id	week_id, brand_id, planned/actual runtime	ShowResult	Separate planned vs result
show_results	result_id	show_id, score, grade, overrun	ShowResult	Mostly ready
segment_results	segment_result_id	result_id, segment_id, score/outcome	SegmentResult	Add stable result id
wrestler_weekly_state	composite	wrestler_id, week_id, momentum/morale/fatigue/heat/trust	current wrestler fields	Add snapshots
injuries	injury_id	wrestler_id, occurred, cleared, severity, description	wrestler fields + notes	Create lifecycle
finance_reports	finance_report_id	show_result_id, attendance, totals	FinanceReport	Add FK
revenue_lines	line_id	report_id, label, amount	revenueBreakdown	Easy
cost_lines	line_id	report_id, label, amount	expenseBreakdown	Easy
social_posts	post_id	result_id, text, author, tone/category	SocialPost	Add result/segment FK

Table/collection	PK	Important fields	Current equivalent	Migration gap
fan_reaction_events	reaction_id	event_id/result_id, audience delta/tone	derived social/audienceHeat	Missing
cpu_brand_results	cpu_result_id	brand_id, week_id, score, notes	RivalBrandWeeklyResult	Normalize out of brand
office_events	office_event_id	brand_id, week_id, deltas, cause event	OfficeMandateEvent	Add cause FK
history_events	event_id	event_type, week_id, source, entity refs, payload	scattered histories	Biggest gap

13. Top 10 Architecture/Data Risks

1. App.tsx still owns too many domain mutations, so simulation rules and UI lifecycle are coupled.
2. Domain IDs use Date.now for segments/rivalries/manual title events, making event causality less deterministic.
3. No unified durable event ledger; histories are scattered by screen/system.
4. Save model has wrapper version but no explicit GameState schema version or migration registry.
5. Segment and SegmentResult are optional bags rather than discriminated unions.
6. Player brand is not a normalized entity; it is implied by GameState fields and player owner strings.
7. CPU rival state is parallel but asymmetric, making deeper rival simulation expensive.
8. Social/finance/history artifacts duplicate names and note text without enough result/segment/event foreign keys.
9. Nested arrays in GameState are fine for localStorage but poor for future database queries and partial updates.
10. Read-model duplication between App.tsx, gameContextReads.ts, and screen read files raises consistency risk.

14. Recommended Next Architecture Slices

Slice	Goal	Likely files	Do not touch	Validation tier	Benefit
Stable domain IDs	Replace Date.now gameplay IDs with deterministic ID helpers	src/game/id.ts, App.tsx, BookingScreen.tsx, smartRundown.ts, socialInboxActions.ts	scoring balance	Small Behavior Slice	Reliable references/history
Booking segment union	Split Segment by type/catalog requirements	types.ts, booking utils, migration	UI redesign	Large System Slice	Safer segment expansion
Result/event normalization	Add resultId/segmentResultId links and common event shape	types.ts, scoring.ts, social/finance reads	AI commentary	Large System Slice	Durable causality
Week history ledger	Add append-only historyEvents generated from show/title/rivalry/injury/finance/market	types.ts, scoring.ts, advanceWeek.ts, migration	screen rewrites	Large System Slice	Better trends/explanations
Save version layer	Add gameSchemaVersion and versioned migration functions	gameStorage.ts, migration.ts, tests	gameplay tuning	Small Behavior Slice	Future-proof saves
App command extraction	Move mutation handlers into src/game/commands/*	App.tsx, new command modules	visual redesign	Read-Only/Small Behavior	Shrinks UI-domain coupling
Brand normalization	Introduce player brand ID/read model without DB	types.ts, seed, market, CPU reads	rival HQ screens	Large System Slice	Database-ready ownership
Social/finance event links	Add result/segment refs to posts/reports	types.ts, social.ts, finance.ts, migration	text pools	Small Behavior Slice	Auditable causality

15. Final Verdict

Is the current architecture simulation-ready? Partially. It can support the current week-to-week loop and several more bounded systems, but not a much deeper simulation without first hardening IDs, events, and command boundaries.

Is the current data model database-ready? No. It is localStorage-snapshot ready, not normalized database ready.

Single highest-leverage architecture fix: introduce a durable, typed week/event ledger with deterministic IDs and explicit links to show results, segment results, wrestlers, titles, rivalries, finance, social, market, office, and injuries.

Do not expand these until the model stabilizes: deeper scouting, editable rival HQ, complex accounting, richer tag/team mechanics, full offseason systems, generated narrative ownership of simulation, and multi-year career pressure.